Making `Ein` a First-Class Value in `Val`

A design proposal for `sky-db`

Status: proposal (plan mode — not yet implemented).

1 Problem

`Ein` (`struct Ein(pub i32)`) exists only as a *subject* type. Whenever an entity is used as a *value* it collapses into a raw `Val::U32`:

- `From<Ein> for Val` → `services::val(ein.to_i32())` → `Val::U32(ein as u32)`
- `datom::add(..., ent("c"))` resolves the temp entity to an `Ein`, then `val(eid)` → `Val::U32`
- in the query layer, `ev_iter` yields `(i32, Val)` and `kb.rs` reifies the subject as `Term::from(e)` (an `i32`) → `Val::U32`

Three distinct concepts are conflated into one type:

Concept	Role	Today	Target
subject entity	subject slot	`Val::U32`	`Ein`
reference to entity	value slot	`Val::U32`	`Val::Ein(Ein)`
literal integer	value slot	`Val::U32`	`Val::U32`

A reference to entity #123 is byte-for-byte indistinguishable from the literal number 123.

2 Goal

Make `Ein` a first-class `Val`: add a distinct variant `Val::Ein(Ein)`, carried end-to-end through *write* → *storage* → *query* → *read*, so an entity reference is a different value from a number, and `Ein` round-trips losslessly (including through snapshots and serialization).

3 Resolved decisions

- **D1 — demote `Attr`, promote `AttrEin`.** `AttrEin` (the attribute *entity id*) becomes the engine's and storage's attribute identity; `Attr` (the ident string) is demoted to an input alias and an output rendering. Attributes are ordinary entities: their `db/ident` and `db/cardinality` are just facts.
- **D2 — lenient now, strict later (a single flip).** All leniency that tolerates `Val::U32` is a migration stopgap. When `Val::Ein` lands, every lenient arm is removed in *one* commit. Nothing stays permanently lenient.
- **D3 — schema-as-data / ref value-types.** Deferred. Contiguous with D1, addressed after the engine migration.

4 Core change: `Val::Ein`

`types/val.rs`:

```
pub enum Val {
    U32(u32),
    U64(u64),
    String(String),
    Bytes(Vec<u8>),
    Ein(Ein),          // NEW
}

impl From<Ein> for Val { fn from(e: Ein) -> Self { Val::Ein(e) } }

impl Val {
    pub fn ein(&self) -> Ein { match self { Val::Ein(e) => *e, _ => panic!("Not an ein") } }
    pub fn try_into_ein(self) -> Option<Ein> { match self { Val::Ein(e) => Some(e), _ => None }
}
```

`_internal/services/val_table.rs` gets a new storage tag so an entity ref hash-conses to a *different* `Vid` than a literal number of the same bits:

```
const VAL_TYPE_EIN: u8 = 20;
// bytes_from_val:  [VAL_TYPE_EIN][ein as u32, 4 big-endian bytes]
// val_from_bytes:  VAL_TYPE_EIN => Val::Ein(Ein(u32_be))
```

This tag is the crux of "first-class": `val(123u32)` and `ein(123)` are distinct values.

5 Query engine: facts as typed triples

The engine moves from attribute-keyed binary atoms to typed three-slot facts. (§10 maps this engine onto the minimal Datalog evaluator it is modeled on.)

5.1 Types

```
pub enum Term<T>               { Var(Var), Lit(T) }
pub enum Binding               { Ein(Ein), Attr(AttrEin), Val(Val) }
pub struct Fact                { e: Ein, a: AttrEin, v: Val }
pub struct Pattern             { e: Term<Ein>, a: Term<AttrEin>, v: Term<Val> }
pub struct Substitution(/* HashMap<Var, Binding> */);
```

`Var` is a plain `&'static str` name; **the slot supplies the type statically**, so no caller-supplied "is this var an ein?" predicate is needed. The only "predicates" are two engine-resident coercers on `Binding`:

```
impl Binding {
    pub fn as_ein(&self) -> Option<Ein> {
        match self {
            Binding::Ein(e)  => Some(*e),
            Binding::Attr(a) => Some(a.ein()),        // an attribute IS an entity
            Binding::Val(Val::Ein(e)) => Some(*e),    // STRICT: only refs deref
            // Val::U32(n) => Some(Ein(n as i32)),    // LENIENT interim - delete at flip
            _ => None,
        }
```

```
    }
    pub fn as_attr(&self) -> Option<AttrEin> {
        match self { Binding::Attr(a) => Some(*a), _ => None }  // no schema at runtime, for now
    }
    pub fn to_val(&self) -> Val {
        match self {
            Binding::Ein(e)  => Val::Ein(e),          // Val::U32 interim
            Binding::Attr(a) => Val::Ein(a.ein()),     // Val::U32 interim
            Binding::Val(v)  => v.clone(),
        }
    }
}
```

5.2 Unification (per slot + coercion matrix)

```
impl Pattern {
    pub fn unify(&self, fact: &Fact) -> Option<Substitution> {
        let mut s = Substitution::new();
        s.bind_subject(&self.e, fact.e)?;   // Lit==Lit, or bind Binding::Ein
        s.bind_attr(&self.a, fact.a)?;      // ... Binding::Attr
        s.bind_value(&self.v, &fact.v)?;    // ... Binding::Val
        Some(s)
    }
}

fn unify_bindings(a: &Binding, b: &Binding) -> Option<Binding> {
    if a == b { return Some(a.clone()); }
    match (a.as_ein(), b.as_ein()) {                        // same entity
        (Some(x), Some(y)) if x == y => return Some(Binding::Ein(x)),
        _ => {}
    }
    match (a.as_attr(), b.as_attr()) {                      // same attribute
        (Some(x), Some(y)) if x == y => return Some(Binding::Attr(x)),
        _ => {}
    }
    None
}
```

`unify_bindings` is invoked only on cross-slot variable reuse. It gives, for free: `Ein(x)` (subject) vs `Val::Ein(x)` (ref value) coerces to `Ein(x)`; and `Attr(a)` (attr slot) vs `Ein(a.ein())` coerces to `Ein`, which is what makes schema-as-data joins fall out of the same rule.

5.3 Grounding (`ground`)

`ground` runs before indexing/unification; it instantiates known bindings into `Lit`s and enforces them as equality constraints. It is where the reverse-deref lives (a value-slot binding reused in the subject slot becomes `Lit(Ein(...))`):

```
trait Coerce: Clone { fn from_binding(b: &Binding) -> Option<Self>; }
```

```
impl Coerce for Ein     { fn from_binding(b: &Binding) -> Option<Ein>     { b.as_ein() } }
impl Coerce for AttrEin { fn from_binding(b: &Binding) -> Option<AttrEin> { b.as_attr() } }
impl Coerce for Val     { fn from_binding(b: &Binding) -> Option<Val>     { Some(b.to_val()) } }

fn ground_term<T: Coerce>(term: &Term<T>, sub: &Substitution) -> Option<Term<T>> {
    match term {
        Term::Lit(l) => Some(Term::Lit(l.clone())),
        Term::Var(v) => sub.get(v)
            .map(|b| T::from_binding(b).map(Term::Lit))
            .unwrap_or(Some(Term::Var(*v))),
    }
}
```

If a bound var cannot coerce into its slot type, `ground` returns `None` — the body atom contributes no facts, giving a *total and local* type error. Heads (derived relations) stay erased; grounding them is total because every `Binding` renders to a `Val`.

5.4 Fact materialization + `facts_stream`

`db_trie` gains typed materializers, keyed on `AttrEin` (with `Schema::cardinality_of` added so cardinality is O(1) by ein):

```
pub fn facts_for_attr<S>(trie, aid: AttrEin, cardinality, order) -> Vec<Fact>;   // AEVT under a
pub fn facts_for_entity<S>(trie, eid: Ein, schema, order) -> Vec<Fact>;          // EAVT under e
pub fn all_facts<S>(trie, schema, order) -> Vec<Fact>;                           // full scan
```

The KB dispatches by which slots are ground:

attribute slot	subject slot	index used
`Lit(aid)`	any	AEVT under `aid`
`Var`	`Lit(eid)`	EAVT under `eid`
`Var`	`Var`	full EAVT scan

This is precisely what folds `AllEins`, `AttrsOfEin`, `EntityFills` (and `AllAttrs`) into plain datalog — no more `unreachable!()` `apply` overrides. Derived relations (arity-N synthetic names like `db::query`, `query_1`, `path`) live in a separate erased space chained in when the attribute does *not* resolve to an `AttrEin`.

6 `Find` migration

6.1 Surface pattern vs engine pattern

`where_` stays ergonomic (no schema available), expressing the attribute as an `Attr` ident:

```
pub struct RawPattern { e: Term<Ein>, a: Term<Attr>, v: Term<Val> }   // what where_ returns
pub struct Pattern    { e: Term<Ein>, a: Term<AttrEin>, v: Term<Val> } // resolved in run_find

fn resolve(p: RawPattern, schema: &Schema) -> Option<Pattern> { /* Attr -> AttrEin */ }
```

Builders: `ein(..)`, `attr(..)`, `val(..)`, `var("..")`, `pattern(e, a, v)`; `Var` is shared across slots, so the *same name* in different slots is the same variable.

6.2 Trait

```
pub trait Find {
    type Output;
    fn select(&self) -> Vec<&'static str>;
    fn where_(&self) -> Vec<RawPattern>;
    fn process(self, pod: &Pod, rows: Vec<Row>) -> Vec<Self::Output>;
}

pub struct Row(HashMap<&'static str, Binding>);   // typed bindings, checked access
```

`process` gains `&Pod` so attribute ids can be resolved back to idents. `run_find` no longer fabricates a dummy `db::query` head — it solves the body to a set of `Substitution`s and projects the selected columns (this is the article's "queries are fresh-head rules," returning the body's bindings directly without materializing the synthetic head):

```
fn run_find(pod, select, where_) -> Vec<Row> {
    let body: Vec<Pattern> = where_.into_iter().filter_map(|p| resolve(p, pod.schema())).collect
    solve_body(pod, &body).into_iter().map(|sub| Row::project(&sub, &select)).collect()
}
```

6.3 The seven impls

impl	body pattern	select	process
`ValsInSlot(e,a)`	`pattern(ein(e), attr(a), var("val"))`	`["val"]`	`to_val()`
`EinsWithAttr(a)`	`pattern(var("ein"), attr(a), var("ignore"))`	`["ein"]`	`as_ein()`
`BindsForAttr(a)`	`pattern(var("ein"), attr(a), var("val"))`	`["ein","val"]`	`as_ein()`, `to_val`
`AllEins`	`pattern(var("ein"), var("a"), var("v"))`	`["ein"]`	`as_ein()`
`AllAttrs`	`pattern(var("a"), attr(db::ident()), var("n"))`	`["a"]`	`as_attr()` → `fi`
`AttrsOfEin(e)`	`pattern(ein(e), var("a"), var("v"))`	`["a"]`	`as_attr()` → `fi`
`EntityFills(e)`	`pattern(ein(e), var("a"), var("v"))`	`["a","v"]`	`as_attr()` → `fi`

`AllAttrs` is expressed as data ("every entity whose `db/ident` exists"), which works only because genesis writes `db/ident` into the same trie as everything else.

7 The single-flip checklist

One migration stopgap, removed wholesale when `Val::Ein` lands:

Surface	interim (now)	final (after `Val::Ein` lands)
`From<Ein> for Val` (write)	`Val::U32(ein as u32)`	`Val::Ein(ein)`
`From<Val> for Ein` (read)	`U32` and `Ein`	`Val::Ein` only (strict)
`From<Val> for Ent` (delegates)	lenient	strict
`Binding::as_ein` (join deref)	accepts `U32`	`Val::Ein` only
`Binding::to_val` (render)	`Ein` → `U32`	`Ein` → `Val::Ein`

After the flip, `Ein::from(Val::U32(...))` is a hard type error — the ambiguity is gone. In the interim, existing `Ein::from(map["ein"])` casts keep working because both the value and the conversion are lenient.

8 Sequencing

1. **Storage / `Val::Ein` outpatient.** Add the variant, the val-table tag, write/read conversions (interim-lenient), and `ein`/`try_into_ein` accessors. Behavior unchanged; refs still stored as `U32` until step 4.
2. **Promote `AttrEin`, demote `Attr`.** Move `AttrEin` to `types`, add `Schema::resolve` → `Option<AttrEin>` and `cardinality_of` (with `by_ein` storing `AttributeDetails`).
3. **Typed query engine.** `Fact`/`Pattern`/`Term<T>`/ `Binding`/`Substitution`; `unify`, `ground`, `facts_stream` (index dispatch). Fold the four `unreachable!()` `Find` impls into `select`/`where_`/`process`.
4. **The flip.** Land `Val::Ein` for real: remove all `Val::U32` lenient arms (checklist) in one commit; refs now persist and return as `Val::Ein`.
5. **Later (deferred).** Schema-as-data and ref value-types (D3).

Steps 1–3 preserve all existing tests; the `program_test` join keeps passing during the interim because refs are still stored as `U32` and `as_ein` is lenient. Step 4 is where refs become strictly typed.

9 Deferred / open questions

- `as_attr` **and schema.** Whether an attribute *reference* (`Val::Ein(attr_ein)`) should be recognized as an attribute — the one place a schema-backed predicate would be needed. Deferred to D3.
- **No value index.** The engine has EAVT/AEVT but no value-oriented index; a ground-*value* query falls to a full scan + filter. A future `AVET`/`VAET` would close this.
- `Row` **typed vs erased** `FindResult`**.** Typed `Row` (recommended) removes the last implicit cast; keeping an erased `Val` result is the minimal alternative.
- `facts_for_entity` **index shape.** Current EAVT recomposition vs a dedicated entity-gated AEVT pass. Kept as-is; revisit for scan efficiency.

10 Relation to *The Essence of Datalog*

This proposal re-expresses the minimal Datalog evaluator from "The Essence of Datalog" (`dodisturb.me/posts/2018-` over an EAV trie. The correspondence:

article	this proposal
`Rule { head, body }`	`Rule` (unchanged; a head is a derived relation)
`Atom { predSym, terms }`	`Pattern` / `Fact` — `predSym` is the attribute, lifted into a typed slot
`Term = Var | Sym`	`Term<T>= Var | Lit(T)`, $T \in \{Ein, AttrEin, Val\}$
`Substitution = [(Term, Term)]`	`Substitution` (var → `Binding`)
`substitute`	`ground`
`unify` (the fact is ground)	`Pattern::unify` + `unify_bindings`
`evalAtom` (substitute, then unify)	`derive_body_atom_subs` (ground → `facts_stream` → unify)
`walk` (fold the body)	`derive_body_subs`
`evalRule` (substitute the head)	`derive_facts`
`immediateConsequence` / `solve` (fixpoint)	`Program::solve` (unchanged fixpoint)
`isRangeRestricted`	`Rule::is_range_restricted` (already present)
query = fresh-head rule	`run_find` returns body bindings directly (equivalent)

Evaluation strategy. The select-before-join order is preserved exactly: ground the already-bound terminals, index the fact base by the ground slots (`facts_stream`), then unify each candidate. That is `evalAtom`'s "substitute, then unify," lifted to a typed, indexed base.

Typed constants. The article uses one untyped constant domain (`Sym String`); here constants are typed (`Ein`/`AttrEin`/`Val`) because storage is an EAV trie of typed values. This is a refinement, not a departure — the article itself gestures at tightening the types (footnote 3). The invariant is retained: a term is either a variable or a constant, and constants compare by identity.

The universal `holds/3` relation. The article runs over one named relation per predicate (`adviser/2`, `name/2`, ...). An EAV store holds every fact in one ternary relation `holds(e, a, v)` (a datom). `Fact { e, a, v }` *is* a row of `holds/3`, and the attribute occupies a term slot rather than the predicate name — which is what collapses `AllAttrs`, `AttrsOfEin`, and `EntityFills` into ordinary rules, and is the schema-as-data move (an attribute is itself an entity). A per-attribute binary view is then just `adviser(X, Y) :- holds(X, adviser_aid, Y)` — i.e. a `Pattern` with a concrete `AttrEin` in the attribute slot.

The one deliberate extension: canonicalization. A subject `Ein(e)` and a reference value `Val::Ein(e)` name the *same entity*, so the article's `Sym == Sym` equality is widened to "compare canonical ids": `Binding::as_ein` projects both onto the shared entity-id domain, and `unify_bindings` compares the projections. This is the cost of making `Ein` a first-class type distinct from `Val` — two spellings of one constant, reconciled at the join — and it is the single place the proposal exceeds minimal Datalog. It is deliberate: it lets a `Val::Ein` value join an `Ein` subject without a silent integer↔entity cast. (The alternative — collapse subject and ref into a single `Val::Ein` constant so join is plain `==` — would be more minimally faithful but loses the compile-time subject/attr/value slot types.)

Termination and range restriction carry over unchanged: the constant domain is finite (eins are 30-bit; values are hash-consed in the val table), `immediateConsequence` is monotone over a finite fact lattice, and `is_range_restricted` guards the fixpoint exactly as in the article.